CodeLab

Online Compiler

Software Documentation Suite

Version:	1.0
Date:	2026-10-10
Author:	Jay Jogane
Stack:	Next.js 16.4.0, React 19.3.0, TypeScript, Tailwind CSS 4, OneCompiler Embed (iframe)

Confidentiality: This document is generated strictly from the repository listed below. No customers, metrics, APIs, or features outside the repository are claimed. Repository: C:/Dev/codelab (root prototype `index.html` + Next.js app in `codelab/`)

Contents

1 Executive Summary **1**
1.1 Problem and solution . . . 1
1.2 Technology and architecture . . . 1
1.3 Maturity and limitations . . . 1

2 Project Overview **1**
2.1 Identity . . . 1
2.2 Repository inventory . . . 2
2.3 Explicit non-goals (not in repository) . . . 2

3 Product Description **2**
3.1 Product identity . . . 2
3.2 Problem and solution . . . 2
3.3 Target audience and personas . . . 2
3.4 Features (implemented) . . . 3
3.5 Non-features . . . 3
3.6 Visual identity and SEO keywords . . . 3

4 Product Requirements Document (PRD) **3**
4.1 Vision and goals . . . 3
4.2 Personas and user stories . . . 3
4.3 Scope . . . 3
4.4 Acceptance criteria (verifiable) . . . 4
4.5 Constraints, dependencies, risks . . . 4

5 Functional Requirements **4**

6 Non-Functional Requirements **5**

7 Technical Requirements **5**
7.1 Explicitly absent . . . 6

8 Software and System Architecture **6**
8.1 System context . . . 6
8.2 High-level design . . . 6
8.3 Low-level design . . . 6
8.4 Backend, database, and API . . . 6

9 Frontend Architecture **7**
9.1 Route tree . . . 7
9.2 Component catalogue . . . 7
9.3 Props contract . . . 7
9.4 Styling approach . . . 7
9.5 State and data fetching . . . 7

10 Integrations **7**
10.1 OneCompiler embed . . . 8
10.2 Resilience notes . . . 8

11 Configuration **8**
11.1 Environment variables . . . 8

12 UI/UX **9**
12.1 Screens . . . 9
12.2 Layout and responsive behavior . . . 9
12.3 States and validation . . . 9
12.4 Accessibility . . . 9

13 Design System **9**

14 Security **10**

15 Performance and Scalability **11**
15.1 Performance . . . 11
15.2 Scalability . . . 11

16 Testing **12**
16.1 Current state . . . 12
16.2 Recommended strategy (RECOMMENDED, not implemented) . . . 12

17 Deployment **12**
17.1 Prerequisites . . . 12
17.2 Build and hosting . . . 12

18 SEO **13**
18.1 Current implementation . . . 13
18.2 Recommended improvements (RECOMMENDED) . . . 13

19 Known Issues **13**

20 Technical Debt **14**

21 Roadmap **15**
21.1 Existing (EXISTING) . . . 15
21.2 Recommended next (RECOMMENDED) . . . 15
21.3 Potential future work (POTENTIAL) . . . 15

22 Development Guidelines **15**
22.1 Setup and commands . . . 16
22.2 Conventions . . . 16
22.3 Definition of done . . . 16

23 Glossary **16**

List of Tables

1 Project identity (evidence-based) . . . 1
2 Implemented features with evidence . . . 3
3 Functional requirements traceable to code . . . 4
4 Non-functional requirements and their implementation state . . . 5
5 Technology stack (actual versions) . . . 5
6 Technologies not present (verified by file search) . . . 6
7 Components (exhaustive) . . . 7
8 OneCompiler integration fact sheet . . . 8
9 Configuration files and their role . . . 8
10 Environment variables . . . 9

11 Actual design values in use 10
12 Security review (evidence-based) 11
13 SEO signals present in the repo 13
14 Verified issues 14
15 Technical debt register 15
16 Glossary (repository-scoped) 16

1 Executive Summary

Status: IMPLEMENTED (what exists) + INFERRED (interpretation, marked inline).

CodeLab is a single-page web front end that embeds the OneCompiler online code editor in a full-viewport window. **Status:** IMPLEMENTED. Evidence: `codelab/src/app/page.tsx` renders only `CodeEditor`; `codelab/src/components/CodeEditor.tsx` renders an `iframe` pointed at `https://onecompiler.com/embed`.

1.1 Problem and solution

The repository prototype (`index.html`, 563 lines) explored a marketing-style landing page (navbar, hero, features) around an embedded compiler, but the shipped Next.js app implements only the compiler window itself. **Status:** IMPLEMENTED. The problem addressed is giving users a zero-install page where they can write and run code; the solution delegates editing, compilation, and execution entirely to the OneCompiler embed. **Status:** INFERRED from the embed URL and page title "CodeLab — Online Compiler" in `codelab/src/app/layout.tsx`.

1.2 Technology and architecture

Next.js 16.4.0 (App Router), React 19.3.0, TypeScript (strict), Tailwind CSS 4. **Status:** IMPLEMENTED. Evidence: `codelab/package.json`, `codelab/tsconfig.json`, `codelab/src/app/globals.css`. There is no backend, database, API route, authentication, test suite, CI pipeline, Dockerfile, or environment configuration in the repository. **Status:** IMPLEMENTED (verified by file enumeration on 2026-10-10).

1.3 Maturity and limitations

Maturity is prototype/demonstration level: four application source files, static rendering, one third-party runtime dependency (OneCompiler). Key limitations: (a) all editor capability depends on the third-party iframe and network access to `onecompiler.com`; (b) no offline support, persistence, user accounts, or usage analytics in the repo; (c) the header status text ("Ready to compile") is static and does not reflect actual compiler state. **Status:** IMPLEMENTED for (a)–(c); see Sections 19 and 20.

2 Project Overview

Status: IMPLEMENTED.

2.1 Identity

Table 1: Project identity (evidence-based)

Item	Value
Name	CodeLab
Subtitle	Online Compiler
Repository layout	Root static prototype `index.html` + Next.js app in `codelab/`
App package	`codelab`, version `0.1.0`, private, `codelab/package.json`
Entry route	`/` from `codelab/src/app/page.tsx`
Document title	"CodeLab — Online Compiler" in `codelab/src/app/layout.tsx`

2.2 Repository inventory

The complete application source (excluding `node_modules/`, `.next/`, `.git/`) is:

- `index.html` — 563-line standalone prototype with inline CSS; contains navbar/hero/features styles but its `body` renders only the compiler section (lines 516–557).
- `codelab/src/app/layout.tsx` — root layout, Geist fonts, metadata, full-viewport body.
- `codelab/src/app/page.tsx` — home route, renders `CodeEditor` only.
- `codelab/src/app/globals.css` — Tailwind import plus default template variables.
- `codelab/src/components/CodeEditor.tsx` — 185-line reusable embed component plus `buildOneCompilerS` URL builder.
- `codelab/package.json`, `next.config.ts`, `tsconfig.json`, `eslint.config.mjs`, `.gitignore` — toolchain configuration.
- `codelab/public/` — default Next.js static assets (`next.svg`, `vercel.svg`, favicons); none are referenced by `page.tsx`. **Status:** IMPLEMENTED (unused assets, see Section 20).

2.3 Explicit non-goals (not in repository)

No backend, database, migrations, API endpoints, authentication, authorization, middleware, background jobs, webhooks, storage, analytics, tests, or deployment manifests exist. These areas are documented as NOT APPLICABLE in their respective sections rather than omitted silently.

3 Product Description

Status: IMPLEMENTED for features; INFERRED where user value is interpreted (marked).

3.1 Product identity

CodeLab is a thin full-viewport wrapper around the OneCompiler editor embed. Product category: online code playground / compiler front end. **Status:** IMPLEMENTED.

3.2 Problem and solution

Problem (INFERRED): users who want to try or share code without installing toolchains need an instantly available editor. Solution (IMPLEMENTED): a single route that fills the viewport with the OneCompiler iframe, framed by a minimal dark window chrome (traffic-light dots, title, static status indicator).

3.3 Target audience and personas

No personas are defined in the repository. **Status:** UNKNOWN. Plausible audiences (INFERRED, not claimed as requirements): students trying small snippets, interviewers sharing a runnable page, documentation authors linking a live editor. These are listed as hypotheses only.

3.4 Features (implemented)

Table 2: Implemented features with evidence

Feature	Evidence
Full-viewport editor	`CodeEditor.tsx` section `h-dvh w-full flex-col`; `page.tsx` and `layout.tsx` bodies use `h-dvh overflow-hidden`
OneCompiler embed	`iframe#onecompiler`, `allow="clipboard-read; clipboard-write"`
Configurable embed URL	`buildOneCompilerSrc` supports `language`, `codeId`, `theme`, `fontSize`, 12 boolean flags, plus full `src` override
Dark window chrome	Tailwind classes `bg-[#0f172a]`, `bg-[#070b12]`, status dot `bg-green-500`
Responsive header	Status cluster hidden on small screens (`max-sm:hidden`), reduced header padding

3.5 Non-features

User accounts, saved sessions, multi-file projects owned by CodeLab, theming beyond the embed `theme`/`fontSize` parameters, and offline execution do not exist in the repository. **Status:** NOT APPLICABLE (not implemented).

3.6 Visual identity and SEO keywords

Visual identity is dark navy (`#070b12` page, `#0f172a` editor) with indigo/cyan accents inherited from the prototype. Metadata title "CodeLab — Online Compiler" and description "Write, compile and run code online with CodeLab." are the only SEO signals. **Status:** IMPLEMENTED. See Section 18.

4 Product Requirements Document (PRD)

Status: INFERRED from implementation (no PRD exists in the repository). Nothing below should be read as a historical requirement; it reconstructs what the code actually satisfies.

4.1 Vision and goals

Vision (INFERRED): the fastest path from link to runnable code in a browser. Goals satisfied by the code: (1) one route, no navigation; (2) editor visible without scrolling on desktop and mobile viewports; (3) zero CodeLab-side code execution to operate.

4.2 Personas and user stories

No personas or stories exist in the repo. **Status:** UNKNOWN. Reconstructed stories (INFERRED): as a visitor, I can open `/` and immediately type and run code (via OneCompiler); as an embedder, I can pass `language`/`fontSize`/hide-flags to tailor the iframe per page.

4.3 Scope

In scope (IMPLEMENTED): full-viewport layout, dark chrome, configurable embed URL, clipboard permissions for the iframe, lazy iframe loading. Out of scope (NOT APPLICABLE): accounts, persistence, collaboration, grading, plagiarism detection, admin tooling.

4.4 Acceptance criteria (verifiable)

1. `npm run build` succeeds (verified 2026-10-10). **Status:** IMPLEMENTED.
2. `/` renders exactly one `section#compiler` containing one `iframe#onecompiler`. **Status:** IMPLEMENTED.
3. No horizontal or vertical gap exists between component and viewport at 360px, 768px, and 1440px widths (visual check recommended; layout classes assert it). **Status:** IMPLEMENTED (code) / RECOMMENDED (manual visual sign-off).
4. With network access to `onecompiler.com`, the iframe loads the editor; without it, the page frame still renders. **Status:** INFERRED (third-party behavior, not mocked in repo).

4.5 Constraints, dependencies, risks

Constraint: all execution capability is delegated to OneCompiler; CodeLab cannot fix editor bugs or guarantee language availability. Dependency: public internet + `onecompiler.com` availability. Risk: embed policy or URL-schema changes break the page without a CodeLab code change. Mitigation (RECOMMENDED): pin and document a known-good embed URL and add a smoke check (see Section 21).

5 Functional Requirements

Status: IMPLEMENTED unless marked otherwise. IDs are documentation-only (no tracker exists in the repo).

Table 3: Functional requirements traceable to code

ID	Requirement	Evidence
FR-01	Render a full-viewport compiler window on `/`	`page.tsx` renders `CodeEditor` inside `h-dvh overflow-hidden` containers
FR-02	Embed the OneCompiler editor via iframe	`CodeEditor.tsx` lines 174–181, `iframe#onecompiler`
FR-03	Default embed hides the remote title and enables code-change events	Defaults `hideTitle=true`, `codeChangeEvent=true`
FR-04	Allow language pre-selection via URL path	`language` prop → `/embed/{language}`
FR-05	Allow preloading saved code via code id	`codeId` prop → `/embed/{language}/{codeId}`
FR-06	Allow editor theme and font-size selection	`theme`, `fontSize` (clamped 8–32 in `clampFontSize`)
FR-07	Allow hiding editor chrome elements	Flags `hideNew`, `hideRun`, `hideStdin`, `hideResult`, etc.
FR-08	Allow full URL override	`src` prop bypasses the builder
FR-09	Display static window title and status	`title` default “Code Lab Compiler”, `statusText` default “Ready to compile”
FR-10	Hide status cluster on narrow screens	`max-sm:hidden` on status container

Out of scope: code persistence, user-triggered run from CodeLab chrome (the component only forwards `listenToEvents` intent to the embed; no `postMessage` sender exists in the repo), and

any server-side compilation. **Status:** NOT APPLICABLE.

6 Non-Functional Requirements

Table 4: Non-functional requirements and their implementation state

Category	Statement	Status and evidence
Responsiveness	No gaps between component and viewport; usable at mobile widths	IMPLEMENTED: `h-dvh`, `flex-1 min-h-0`, reduced mobile header padding
Performance	First paint should be dominated by framework + one iframe	INFERRED: static route, `loading="lazy"` on iframe; no measurements in repo
Accessibility	Title and landmark labeling for the editor region	PARTIAL: `section aria-label`, iframe `title="Online Code Compiler"`; no keyboard/focus management of remote content possible
Compatibility	Modern browsers with iframe + clipboard permissions	IMPLEMENTED: standard iframe attributes; no polyfills in repo
Security	No CodeLab-side secret handling or code execution	IMPLEMENTED: no secrets, no API routes; residual iframe risks in Section 14
Maintainability	Single reusable component with typed props	IMPLEMENTED: `CodeEditor.tsx` + `buildOneCompilerSrc`
Availability	Page frame renders even if embed is unreachable	INFERRED: no error boundary exists; frame renders, iframe area stays empty (RECOMMENDED improvement)

7 Technical Requirements

Status: IMPLEMENTED (toolchain table verified against manifests on 2026-10-10).

Table 5: Technology stack (actual versions)

Category	Technology	Evidence
Framework	Next.js 16.4.0 (App Router)	`codelab/package.json`; `next.config.ts` enables `cacheComponents`, partial prefetching
UI runtime	React 19.3.0, react-dom 19.3.0	`codelab/package.json`
Language	TypeScript 5, strict mode	`tsconfig.json` (`strict:true`, `jsx:react-jsx`, path alias `@/*`)
Styling	Tailwind CSS 4 via `@tailwindcss/turbopack`	`package.json` devDependencies; `globals.css` `@import "tailwindcss"`
Fonts	Geist + Geist Mono via `next/font`	`layout.tsx`
Lint	ESLint 9 + `eslint-config-next`	`eslint.config.mjs`, script `npm run lint`
Runtime embed	OneCompiler iframe embed	`CodeEditor.tsx` `EMBED_BASE`

7.1 Explicitly absent

Table 6: Technologies not present (verified by file search)

Area	Status
Backend / API routes	NOT APPLICABLE — no `route.ts`, no server actions
Database / ORM / migrations	NOT APPLICABLE
Authentication / sessions	NOT APPLICABLE
State management library	NOT APPLICABLE (props only)
Test runner	NOT APPLICABLE (no test files, scripts, or config)
CI/CD, Docker, IaC	NOT APPLICABLE
Env configuration	NOT APPLICABLE (no `.env*` files; `.gitignore` merely ignores them)

8 Software and System Architecture

Status: IMPLEMENTED. This section merges system architecture, high-level design, and low-level design because the system is small enough that separate documents would repeat each other.

8.1 System context

The deployed unit is a static Next.js route served to a browser; the browser then loads the OneCompiler application in an iframe. CodeLab executes no user code itself.

Listing 1: System context (text diagram; Mermaid source in diagrams/system-context.mmd)

```
Browser -> Next.js static route "/" (layout + page + CodeEditor chrome)
Browser -> https://onecompiler.com/embed... (iframe, editor + runner)
CodeLab server -x- user code (never touches it)
```

8.2 High-level design

1. **Route layer:** `src/app/layout.tsx` (html/body shell, fonts, metadata) composes `src/app/page.tsx` (route `/`).
2. **Component layer:** `page.tsx` renders one `CodeEditor` with default props.
3. **URL-builder layer:** `buildOneCompilerSrc` maps typed props to an embed path plus query string; `src` bypasses it.
4. **Presentation layer:** Tailwind utility classes implement the dark full-viewport window; `globals.css` contributes only the Tailwind import and unused template variables.

8.3 Low-level design

`CodeEditor` is a React Server Component (no `"use client"` directive) with no hooks, context, or event listeners. Data flow is one-way: props → `iframeSrc` string → `iframe src` attribute. `clampFontSize` enforces the documented 8–32 range. `language`/`codeId` are sanitized by stripping `/` characters before interpolation. There are no classes, stores, or services.

8.4 Backend, database, and API

Status: NOT APPLICABLE. No server components fetch data, no route handlers exist, and no schema or migration files are present. Any future backend must be documented separately; this suite does not invent one.

9 Frontend Architecture

Status: IMPLEMENTED.

9.1 Route tree

One route: `/` (`src/app/page.tsx`). No nested layouts, loading states, error boundaries, or not-found customization beyond the framework default. The App Router `LayoutProps<"/">` typing in `layout.tsx` reflects Next.js 16 conventions.

9.2 Component catalogue

Table 7: Components (exhaustive)

Component	Responsibility
`RootLayout` (`layout.tsx`)	html lang, Geist font variables, `metadata` (title, description, favicon), full-viewport flex body
`Home` (`page.tsx`)	Full-height background container (`bg-[#070b12]`) rendering one `CodeEditor`
`CodeEditor` (`components/CodeEditor.tsx`)	Window header (dots, title, status) + responsive iframe body; URL builder

9.3 Props contract

`CodeEditorProps` exposes `title`, `statusText`, `id`, `className`, `src`, `language`, `codeId`, `theme`, `fontSize`, and 12 embed flags. Defaults reproduce the legacy prototype URL (`theme=dark`, `hideTitle`, `codeChangeEvent`). Note an inconsistency documented in Section 19: `buildOneCompilerSrc` defaults `availableLanguages` and `listenToEvents` to `true` while the component defaults them to `false`, so direct helper calls and default component renders produce different query strings.

9.4 Styling approach

Utility-first Tailwind classes co-located with markup; no CSS modules, no design-token layer. Arbitrary values (`bg-[#0f172a]`, `h-dvh`, `max-sm:hidden`) carry the visual system. `globals.css` still ships the `create-next-app` light/dark template variables, which are inert because page-level classes override the body background.

9.5 State and data fetching

None. **Status:** NOT APPLICABLE (no hooks, no context, no fetch, no cache configuration beyond framework defaults in `next.config.ts`).

10 Integrations

Status: IMPLEMENTED (one integration).

10.1 OneCompiler embed

Table 8: OneCompiler integration fact sheet

Item	Detail
Type	Third-party iframe embed; editor and code runner hosted by OneCompiler
Base URL	`https://onecompiler.com/embed` (`EMBED_BASE` in `CodeEditor.tsx`)
DOM contract	`iframe#onecompiler`, `title="Online Code Compiler"`, `loading="lazy"`, `allow="clipboard-read; clipboard-write"`
Configuration surface	Path (`language`, `codeId`) + query flags (Section 5, FR-04–FR-08)
Programmatic API	Only intent flags today: `codeChangeEvent` (embed posts code changes) and `listenToEvents` (embed accepts incoming messages). No `window.onmessage` receiver and no `postMessage` sender exist in this repo.
Docs reference	`https://onecompiler.com/apis/embed-editor` (public embed options)

10.2 Resilience notes

If `onecompiler.com` is unreachable, the CodeLab chrome still renders with an empty editor area; no fallback message or retry exists. **Status:** IMPLEMENTED (behavior) / RECOMMENDED (fallback UI, see Section 21). No other integrations (analytics, auth providers, storage, email) exist. **Status:** NOT APPLICABLE.

11 Configuration

Status: IMPLEMENTED.

Table 9: Configuration files and their role

File	Role
`codelab/next.config.ts`	Enables `cacheComponents` and partial prefetching; Turbopack rule routing `*.css` through `@tailwindcss/turbopack`
`codelab/tsconfig.json`	Strict TS, `ES2017` target, bundler resolution, `@/*` → `./src/*` alias (used by `page.tsx` import)
`codelab/eslint.config.mjs`	Flat config extending `core-web-vitals` + TypeScript presets; ignores build outputs
`codelab/.gitignore`	Ignores `node_modules`, `.next`, `out`, `build`, `.env*`, Vercel metadata

11.1 Environment variables

None are read or required. **Status:** NOT APPLICABLE. The table below records the empty set explicitly so Word readers are not left guessing.

Table 10: Environment variables

Variable	Required	Used by	Sensitive
(none)	—	—	—

12 UI/UX

Status: IMPLEMENTED.

12.1 Screens

Exactly one screen: the editor window on `/`. There is no navigation, footer, dialog, or form owned by CodeLab. The prototype `index.html` styled a navbar, hero, badge, features grid, and footer, but none of those elements are rendered by the Next.js app; they are historical context only.

12.2 Layout and responsive behavior

The section is a vertical flex column at `h-dvh`: a shrink-wrapped header plus a `flex-1 min-h-0` iframe region. On viewports under 640px the status cluster disappears (`max-sm:hidden`) and header padding shrinks, preserving editor height. The iframe fills remaining space at all widths; no max-width container constrains it (the prototype's `min(1400px, ...)` shell was removed when the component was fitted to the viewport).

12.3 States and validation

No loading, empty, error, or form-validation states exist in CodeLab chrome. The `statusText` prop (default "Ready to compile") is static text, not a live reflection of embed readiness. **Status:** IMPLEMENTED (static) / RECOMMENDED (drive it from embed events or remove it to avoid implying liveness).

12.4 Accessibility

Implemented: landmark `section` with `aria-label`, descriptive iframe `title`, decorative dots hidden via `aria-hidden`. Not implemented: focus-visible styles, reduced-motion handling, or keyboard shortcuts (all editor interaction lives inside the cross-origin iframe and cannot be managed here). **Status:** PARTIAL.

13 Design System

Status: IMPLEMENTED (values extracted verbatim from code; no Figma or token file exists).

Table 11: Actual design values in use

Token	**Value and location**
Page background	`#070b12` in `page.tsx` (prototype used the same base plus indigo/cyan radial gradients)
Editor surface	`#0f172a` header (85% opacity) and iframe surround in `CodeEditor.tsx`
Borders	`white/[0.08]` hairline under header
Dots	`bg-slate-600`, 10px circles; status dot `bg-green-500` with green glow shadow
Typography	Title 14px semibold slate-50; status 12px slate-400; body Geist sans via `next/font` (prototype used Inter)
Header metrics	`px 18px / py 15px` desktop, `p-3` mobile; backdrop blur 14px
Radius | Shadows | Motion	Full-bleed: no radius or shadow on the fitted component (prototype had 16px radius + deep shadow on its centered card)
Breakpoints	Tailwind defaults; explicit adaptations at `max-sm` (640px)

There is no button, input, card, modal, table, or icon subsystem in the app; documenting one would be fabrication. **Status:** NOT APPLICABLE beyond the window chrome above.

14 Security

Status: findings are IMPLEMENTED observations; recommendations are marked RECOMMENDED.

Table 12: Security review (evidence-based)

Finding	Severity	Evidence, impact, recommendation
No secrets in repo	Informational	No `.env` files, tokens, or keys found. Impact: none. Recommendation: keep it so; add secret scanning if a backend is ever added.
Cross-origin iframe	Medium	`iframe` loads full OneCompiler app. Impact: CodeLab inherits the embed's XSS/sandbox posture; CodeLab cannot audit remote JS. Recommendation (RECOMMENDED): add `sandbox` allow-list review and a content-security-policy frame-src pin in future hosting config.
Clipboard permissions	Low	`allow="clipboard-read; clipboard-write"`. Impact: editor copy/paste works; a compromised embed could read clipboard on user gesture. Recommendation: keep only if needed; document the trade-off.
No input validation surface	Low	CodeLab takes no user input itself (only props). Impact: injection risk is confined to `language`/`codeId` interpolation, which strips slashes. Recommendation: retain the sanitization; add unit tests for the builder.
Missing security headers	Low	No `headers()` config in `next.config.ts`. Impact: hosting defaults apply. Recommendation (RECOMMENDED): set `frame-ancestors`, `X-Content-Type-Options` at deploy time.

Authentication, authorization, RLS, encryption, upload handling, and audit logging are NOT APPLICABLE: the features do not exist.

15 Performance and Scalability

Status: INFERRED (no measurements, budgets, or load tests exist in the repository).

15.1 Performance

The page ships a static route plus React runtime plus one lazily-loaded third-party iframe. Expected cost centers: (1) Next.js JS bundle, (2) Geist font loading, (3) OneCompiler app inside the iframe, which dominates interactive readiness. Implemented mitigations: static prerendering, `loading="lazy"` on the iframe, no images or client JS of our own. Not implemented: bundle analysis, `next/image` usage (irrelevant—no images rendered), or Web Vitals reporting.

15.2 Scalability

CodeLab-side scaling is trivially horizontal: the route is static with no session state, so any static host or CDN can serve it. Effective scalability is bounded by OneCompiler's availability and rate limits, which are outside this repository. No caching, queueing, or database scaling considerations apply. **Status:** NOT APPLICABLE beyond CDN static hosting.

16 Testing

Status: NOT APPLICABLE (no test infrastructure exists) — documented explicitly rather than invented.

16.1 Current state

No test runner, script, or spec file is present: `package.json` defines only `dev/build/start/lint`; file search finds no `*.test.*` or `*.spec.*`; no Playwright/Cypress/Vitest/Jest configuration exists. The only automated quality gate is `eslint` plus the TypeScript check inside `next build` (both passing at the time of analysis).

16.2 Recommended strategy (RECOMMENDED, not implemented)

1. **Unit:** tests for `buildOneCompilerSrc` (path joining, flag serialization, `fontSize` clamping, slash stripping) and the helper/component default mismatch noted in Section 19.
2. **Component:** render test asserting one `section#compiler` with one iframe and correct `src` for given props.
3. **E2E smoke (optional):** visit `/`, assert the frame renders; stub or allow-list the third-party iframe per environment policy.
4. **Manual QA checklist:** viewport-gap check at 360/768/1440px; embed loads with network; graceful empty editor area without network; clipboard copy/paste inside the embed.

17 Deployment

Status: IMPLEMENTED for local commands; INFERRED for hosting (no hosting config in repo).

17.1 Prerequisites

Node.js with npm (versions not pinned in repo — RECOMMENDED: pin in `.nvmrc` or `packageManager` field). Install from `codelab/`:

Listing 2: Verified commands (from package.json scripts)

```
cd codelab
npm install
npm run dev    # local development
npm run lint   # quality gate
npm run build  # production build + type check
npm start      # serve the production build
```

17.2 Build and hosting

`next build` statically prerenders `/` (verified 2026-10-10 output: "Static prerendered as static content"). No Dockerfile, CI workflow, or platform manifest exists, so any host (Vercel, Netlify, self-hosted Node, static export) is an operator choice, not a repository fact. No database migrations, rollback scripts, or production env configuration exist. **Status:** NOT APPLICABLE.

18 SEO

Status: IMPLEMENTED (current) / RECOMMENDED (improvements). AEO and GEO have no dedicated implementation and are NOT APPLICABLE beyond the general semantic-HTML notes below.

18.1 Current implementation

Table 13: SEO signals present in the repo

Signal	Evidence
Title + description	`layout.tsx` metadata: "CodeLab — Online Compiler" / "Write, compile and run code online with CodeLab."
Language + favicon	`<html lang="en">`, `icons.icon=/favicon.ico`
Semantic landmark	`section#compiler` with `aria-label`, iframe `title`

Missing (verified absent): sitemap, `robots.txt`, canonical URLs, Open Graph/Twitter cards, JSON-LD structured data, and indexable content beyond the chrome (the editor lives in a cross-origin iframe and contributes no crawlable text). The prototype `index.html` had richer marketing copy (hero/features) that was not carried into the app.

18.2 Recommended improvements (RECOMMENDED)

Add OG/Twitter meta, a `robots.txt` + sitemap for the single route, canonical URL once a domain is chosen, and—if search traffic matters—a server-rendered content section describing supported languages, because the current route offers crawlers almost no text.

19 Known Issues

Status: IMPLEMENTED observations (verified in code on 2026-10-10).

Table 14: Verified issues

Issue	Severity	Location, impact, recommendation
Helper/component default mismatch	Medium	`CodeEditor.tsx`: `buildOneCompilerSrc` defaults `availableLanguages=true`, `listenToEvents=true`; the component defaults both to `false`. Impact: documentation examples calling the helper directly produce a different URL than a default render. Recommendation: align the defaults and add unit tests.
Static status text	Low	`statusText="Ready to compile"` never changes. Impact: implies liveness the page does not have. Recommendation: drive from embed events or relabel as static chrome.
Misplaced `<link>` in layout	Low	`layout.tsx` renders `<link rel="shortcut icon">` inside the component body rather than via metadata only. Impact: works but is non-idiomatic and duplicates `icons.icon`. Recommendation: remove the manual tag.
Unused template styling/assets	Low	`globals.css` light/dark template variables and `public/*.svg` are unreferenced. Impact: dead weight, minor confusion. Recommendation: prune or adopt.
No embed fallback	Medium	No error UI when `onecompiler.com` is unreachable. Impact: blank editor area. Recommendation: detect iframe failure and show guidance.

20 Technical Debt

Status: IMPLEMENTED observations; priority is the author's judgment (marked).

Table 15: Technical debt register

Item	Priority	Detail
Untested URL builder	High	`buildOneCompilerSrc` is the only logic in the app and has no tests. Pay down with unit tests (Section 16).
Default mismatch (Sec. 19)	High	Same fix as above; decide the canonical defaults once.
Dead template code	Low	`globals.css` variables, unused `public` SVGs.
Prototype drift	Medium	`index.html` (563 lines, marketing sections) and the app diverged; keep the file as reference or archive it so future readers are not misled.
No pinning of embed contract	Medium	Embed URL schema is assumed from public docs; no version pin or smoke test.
Missing headers/CSP	Low	No security-header configuration (Section 14).

21 Roadmap

Status: all items RECOMMENDED or POTENTIAL unless marked EXISTING. Nothing below is a committed plan (no tracker exists in the repo).

21.1 Existing (EXISTING)

Full-viewport embed, configurable OneCompiler URL builder, dark chrome, responsive header, lint + type-check gates.

21.2 Recommended next (RECOMMENDED)

1. Align `buildOneCompilerSrc` and component defaults; add unit + render tests.
2. Add an embed-failure fallback message with retry guidance.
3. Remove the manual favicon `<link>` and prune unused template CSS/assets.
4. Pin a known-good embed URL in documentation and add a periodic smoke check against `https://onecompiler.com/apis/embed-editor`.
5. Add OG/Twitter meta, `robots.txt`, and sitemap if search presence is desired.
6. Pin Node/npm versions and document them.

21.3 Potential future work (POTENTIAL)

Optional, only if product direction demands: language presets UI, `postMessage` bridge exposing code-change/run events to CodeLab chrome, multi-tab editors, shareable links encoding `language`/`codeId`, and a self-hosted editor to remove the third-party runtime dependency. Each would require new architecture, security, and testing documentation.

22 Development Guidelines

Status: IMPLEMENTED (workflow as configured) + RECOMMENDED (conventions to adopt). This section doubles as the contribution guide; no separate CONTRIBUTING file exists in the

repo.

22.1 Setup and commands

Work from `codelab/`. Path alias `@/*` maps to `./src/*` (`tsconfig.json`); `page.tsx` already uses `@/components/CodeEditor`. Run `npm run lint` and `npm run build` before opening a change; both must pass.

22.2 Conventions

- Keep `CodeEditor` presentational: props in, iframe `src` out; no client state unless a `postMessage` bridge is explicitly designed and documented.
- Keep the embed builder as the single place that knows the OneCompiler URL schema; never hand-concatenate embed URLs elsewhere.
- Preserve the full-bleed contract: no page-level margins, paddings, or max-width wrappers around the editor without updating Sections 12 and 13.
- Prefer Tailwind utilities over new CSS files; the prototype `index.html` is reference-only—do not reintroduce its global reset into the app.
- Do not add backend, auth, or tracking without a matching documentation update and security review.

22.3 Definition of done

Build + lint green, viewport-gap check at 360/768/1440px, embed loads with network access, no secrets added, and any prop/flag change reflected in Section 5 and the component docblock.

23 Glossary

Table 16: Glossary (repository-scoped)

Term	Meaning
CodeLab	This project: a full-viewport front end for an embedded online compiler.
OneCompiler embed	Third-party editor/runner loaded in `iframe#onecompiler` from `onecompiler.com`.
`buildOneCompilerSrc`	Helper in `CodeEditor.tsx` mapping props to an embed URL.
App Router	Next.js routing model used here (`src/app/layout.tsx`, `src/app/page.tsx`).
Static prerender	Build-time HTML generation; `/` is served as static content.
`h-dvh`	CSS dynamic viewport height unit used for the full-bleed layout.
NOT APPLICABLE	Classification meaning the area does not exist in this repository.
INFERRED	Classification meaning a reasonable interpretation of the code, not a repo-stated fact.
RECOMMENDED	Classification meaning a suggested improvement, not implemented functionality.